

Sistema de Gestión de Videojuegos

Introducción:

Eres el encargado de diseñar un sistema que gestione la biblioteca de videojuegos de una tienda online. La tienda tiene varios tipos de videojuegos: algunos son físicos y otros son digitales. Además, cada juego tiene su precio, género y tipo de consola.

Objetivo:

Implementa un sistema que permita gestionar la información de los videojuegos, diferenciando entre videojuegos físicos y digitales, y que pueda calcular el precio final de los videojuegos dependiendo de su tipo y otros factores.

Desafío:

Desarrolla el sistema aplicando los principios de Programación Orientada a Objetos: abstracción, encapsulamiento, herencia, polimorfismo e interfaces. Tendrás que pensar cómo organizar la información de los videojuegos y cómo aplicar los conceptos de POO para hacer el sistema eficiente.

Requisitos:

- Gestiona videojuegos físicos y digitales.
- Los videojuegos físicos incluyen un costo de envío adicional.
- Los videojuegos digitales pueden tener un descuento en ciertas plataformas.
- Usa herencia, interfaces y polimorfismo para modelar el sistema de manera efectiva.

Pistas para resolver el problema:

1. Primera pista (si no sabes cómo empezar):

Piensa en los atributos y comportamientos comunes que tienen todos los videojuegos. ¿Cómo podrías representarlos en una clase general? Un posible enfoque sería crear una clase abstracta Videojuego que represente estos elementos comunes, como el título, la consola y el precio base.

2.Segunda pista (si te cuesta diferenciar entre tipos de videojuegos):

Considera cómo se pueden comportar de manera diferente los videojuegos físicos y digitales. ¿Cómo podría la clase VideojuegoFísico agregar el costo de envío y cómo podría VideojuegoDigital aplicar un descuento? Aquí es donde entra en juego la herencia.

3. Tercera pista (si no sabes cómo usar interfaces):

Los videojuegos pueden pertenecer a diferentes géneros (aventura, acción, etc.). ¿Cómo podrías hacer que ciertos videojuegos pertenezcan a múltiples géneros de manera flexible? Una **interfaz** podría ayudarte a representar este comportamiento adicional.

4. Cuarta pista (si no entiendes cómo aplicar polimorfismo):

Piensa en cómo podrías tener una lista de videojuegos (`List<Videojuego>`) y utilizar el mismo método para calcular el precio final de cada uno, sin preocuparte de si es físico o digital. Esto es polimorfismo en acción: todos los videojuegos deben implementar el método `calcularPrecioFinal()`, pero el cálculo dependerá de si son físicos o digitales.

Ejemplo para inspirarte :

En este ejemplo, The Legend of Zelda es un juego físico con un costo de envío de 5 euros, mientras que Among Us es un juego digital con un descuento del 20%. Tu sistema debe calcular el precio final correctamente en cada caso.

Guía de ayuda (para cuando te atasques)

Si en algún momento sientes que te has quedado atascado, hay un documento separado con más pistas y explicaciones sobre los conceptos clave. Solo recurre a esta guía si lo necesitas, ya que el objetivo es que trates de resolver el problema por ti mismo.

Guía de ayuda para cuando te atascas

Pistas adicionales:

1. Clase abstracta Videojuego

Piensa en crear una clase abstracta Videojuego que tenga atributos como titulo, consola y precioBase. También puede tener un método abstracto calcularPrecioFinal() que se implementará en las clases que hereden de esta clase.

2. Clases específicas: VideojuegoFisico y VideojuegoDigital

Estas clases deben heredar de Videojuego y cada una debe implementar su propio método calcularPrecioFinal(). En VideojuegoFisico, puedes sumar el costo de envío al precio base, mientras que en VideojuegoDigital puedes aplicar un descuento al precio.

3. Uso de interfaces

Los videojuegos pueden pertenecer a múltiples géneros (aventura, acción, deportes, etc.). Tal vez podrías usar una interfaz Genérico con métodos que definan este comportamiento, de modo que los videojuegos puedan implementar varios géneros si es necesario.

4. Polimorfismo

El polimorfismo te permite tratar a todos los videojuegos de manera uniforme, incluso si se comportan de manera diferente. Por ejemplo, puedes tener una lista de videojuegos y llamar al método calcularPrecioFinal() en cada uno sin preocuparte si es físico o digital.

Entregable:

Sube tu código Java junto con un documento donde expliques cómo aplicaste los principios de POO para resolver el problema.

Criterios de evaluación:

- Aplicación correcta de los conceptos de POO.
- Uso adecuado de herencia, polimorfismo e interfaces.
- Estructura y claridad del código.

Con este enfoque más familiar, tus alumnos tendrán un tema más cercano y podrán centrarse mejor en los conceptos de POO mientras crean un sistema que puede resultarles más interesante.

Parte 2 Consolas

Consolas

Las consolas Todas se podran, encender, apagar y jugar juego, la plataforma y decidir si un juego pertenece o no a su plataforma y por tanto se puede instalar.

Tendremos PlayStation, Xbox, Switch.

En todas las consolas podremos instalar juegos de su plataforma, para tener la lista de juegos disponibles para jugar.

En caso de no ser compatibles se lanzará una excepción personalizada.

Requisitos:

1. Consolas disponibles:

- Tendrás tres tipos de consolas: PlayStation, Xbox, y Switch.
- Cada consola debe tener la capacidad de **encenderse, apagarse, instalar juegos y jugar** a los juegos instalados.

2. Juegos compatibles:

- Se podrán instalar en cada consola únicamente los juegos que sean compatibles con su plataforma. Los juegos pueden ser **físicos o digitales**.
- Si un juego es incompatible con la consola, se debe lanzar una **excepción personalizada** indicando el error.

3. Videojuegos:

- Los videojuegos pueden ser de dos tipos: **físicos o digitales**. Ambos tipos de videojuegos deben ser subclases de una clase abstracta **VideoGame**.
- Cada juego debe tener un método para verificar si es compatible con una consola antes de ser instalado.
- Los videojuegos deben contener información básica como el **título** y el **género**.

4. Excepción personalizada:

- Implementa una **excepción personalizada** llamada **JuegoNoCompatibleException** que se lanzará cuando un juego no sea compatible con la plataforma de la consola en la que se intenta instalar.

Especificaciones:

1. Interfaz Console:

- Define una interfaz Console con los siguientes métodos:
- void switchOn(): Método para encender la consola.
- void switchOff(): Método para apagar la consola.
- void installGame(VideoGame game): Método para instalar un juego en la consola. Antes de instalar el juego, verifica si es compatible con la plataforma. Si no lo es, lanza la excepción JuegoNoCompatibleException.
- void playGame(): Método para jugar al primer juego instalado en la consola.
- String getPlataforma(): Método que devuelve el nombre de la plataforma de la consola (ej: "PlayStation", "Xbox", "Switch").

2. Clases de consolas:

- Implementa las clases PlayStation, Xbox y Switch, que implementan la interfaz Console.
- Cada consola debe almacenar una lista de juegos instalados. Solo los juegos compatibles pueden ser instalados.

3. Clase abstracta VideoGame:

- Define una clase abstracta VideoGame con los atributos title (String) y genre (String), y un método abstracto boolean isCompatible(Console console) que debe ser implementado por sus subclasses.

4. Subclases de VideoGame:

- Implementa las subclases VideojuegoPhysic y VideojuegoDigital, que deben implementar el método isCompatible para verificar si el juego puede instalarse en la consola correspondiente.

5. Excepción personalizada:

- Implementa la clase JuegoNoCompatibleException que extiende de Exception. Esta excepción será lanzada cuando se intente instalar un juego en una consola no compatible.